# GUÍA DE LABORATORIO

| INFORMACIÓN BÁSICA | | | | | |
|---|---|---|---|---|---|
| **ASIGNATURA:** | *Tecnología de Objetos* | | | | |
| **TÍTULO DE LA PRÁCTICA:** | *Laboratorio 04* | | | | |
| **NÚMERO DE PRÁCTICA:** | 4 | **AÑO LECTIVO:** | *2026* | **NRO. SEMESTRE:** | *VI* |
| **TIPO DE PRÁCTICA:** | **INDIVIDUAL** | | | | |
| | **GRUPAL** | **MÍNIMO DE ESTUDIANTES** | ---------- | **MÁXIMO DE ESTUDIANTES** | *2* |
| **FECHA INICIO:** | *11/10/2026* | **FECHA FIN:** | *13/10/2026* | **DURACIÓN:** | *2 horas* |
| **INTEGRANTE (s):**<br>*Chipana Jeronimo German Arturo*<br>*Baca Calsin Leonardo Juan José* | | | | | |
| **DOCENTE(s):**<br>*Edith Giovanna Cano Mamani* | | | | | |

## CONTENIDO DE LA GUÍA

**I. SOLUCIÓN**

Repositorio:

https://github.com/ChipanaGerman/TecnologiaDeObjetos-Lab-B/tree/main/Lab4

## INTRODUCCIÓN

El framework Qt es un conjunto de bibliotecas multiplataforma en C++ ampliamente utilizado para el desarrollo de software tanto en entornos de consola como en aplicaciones gráficas de usuario (GUI). La arquitectura de Qt se basa en dos pilares fundamentales: la gestión automática de memoria mediante una jerarquía de objetos y el mecanismo de comunicación desacoplado conocido como **Signals & Slots**. En este laboratorio se exploran desde los tipos de datos primitivos propios de Qt hasta el diseño modular de interfaces gráficas reactivas.

## OBJETIVOS

### Objetivo General

- Comprender y aplicar los principios fundamentales del SDK de Qt y la programación orientada a objetos en C++, dominando la gestión de memoria jerárquica con QObject y la comunicación entre componentes mediante Señales y Ranuras (Signals & Slots) tanto en consola como en entorno gráfico.

### Objetivos Específicos

- Manipular cadenas y funciones de depuración utilizando QString y qDebug().
- Verificar el ciclo de vida de los objetos y la liberación en cascada de memoria que proporciona la clase base QObject.
- Implementar emisores y receptores de eventos personalizados conectando señales y slots.
- Diseñar formularios gráficos utilizando Qt Designer y vincular elementos visuales (Widgets) con la lógica de negocio.

## MARCO CONCEPTUAL

- **QObject:** Clase base de la mayoría de componentes de Qt. Proporciona el árbol jerárquico de objetos (relación padre-hijo) y el soporte para el sistema de metadatos.

- **Gestión de Memoria en Árbol (Parent-Child):** Cuando a un QObject se le asigna un objeto padre, el padre asume la propiedad del hijo. Si el padre es destruido (delete), todos sus hijos son liberados automáticamente de la memoria, evitando fugas (*memory leaks*).
- **Señales (Signals):** Notificaciones emitidas por un objeto cuando ocurre un cambio de estado o evento. No tienen implementación en código (.cpp), solo se declaran.
- **Ranuras (Slots):** Funciones miembros ordinarias que se ejecutan como respuesta directa a la activación de una señal conectada.
- **Macro Q_OBJECT:** Macro obligatoria en toda clase que declare señales, slots o haga uso de introspección/meta-objetos en Qt. Activa el compilador de meta-objetos (*moc*).

**Problema Resuelto:**

**Actividad A: Tipos primitivos, QString y qDebug**

```cpp
#include <QCoreApplication>
#include <QDebug>
#include <QString>

int main(int argc, char *argv[])
{
    QCoreApplication a(argc, argv);

    int x = 47;
    double y = 3.2478;
    char t = 'O'; // En el PDF dice 'Ok' que es un multi-character warning en C++; 'O' es formal.

    // Formateo posicional de cadenas con .arg()
    QString qtcadena = QString("%1 %2 %3").arg(t).arg(y).arg(x);

    // Conversión a std::string y salida por qDebug()
    std::string cadena = qtcadena.toStdString();
    qDebug() << cadena.c_str();
    qDebug() << qtcadena;

    // Métodos utilitarios de QString
    qtcadena = "arequipa ciudad blanca 2021";
    qDebug() << "Contiene 'ciudad':" << qtcadena.contains("ciudad"); // Retorna true (1)
    qDebug() << "Posición de '2021':" << qtcadena.indexOf("2021");   // Retorna el índice numérico

    return 0;
}
```

```
Lab01_Primitivas
23:03:31: Starting C:\Users\User\OneDrive\Documentos\Lab01_Primitivas\build\Desktop_Qt_6_12_0_MinGW_64_bit_Debug\Lab01_Primitivas.exe...
O 3.2478 47
"O 3.2478 47"
Contiene 'ciudad': true
Posición de '2021': 23
23:03:32: The command "C:\Users\User\OneDrive\Documentos\Lab01_Primitivas\build\Desktop_Qt_6_12_0_MinGW_64_bit_Debug\Lab01_Primitivas.exe" finished successfully.
```

Se evidenció el uso de QString frente a las cadenas clásicas de C++. El método .arg() sustituye los comodines %1, %2, etc. en tiempo de ejecución de manera segura y tipada. Asimismo, se utilizó qDebug() como canal de depuración estándar de Qt. Se constató que .contains() evalúa pertenencia retornando un booleano y .indexOf() localiza la posición base cero del substring.

**Actividad B: Control de memoria con QObject**

```cpp
#include <QCoreApplication>
#include <QObject>
#include <QString>
#include <QDebug>

class Adulto : public QObject
{
private:
    QString nombre;

public:
    explicit Adulto(QObject *base = nullptr) : QObject(base) {}

    void setNombre(const QString &nomb) {
        nombre = nomb;
    }

    QString getNombre() const {
        return nombre;
    }

    ~Adulto() override {
        qDebug() << "..objeto destruido..." << nombre;
    }
};

int main(int argc, char *argv[])
{
    QCoreApplication a(argc, argv);

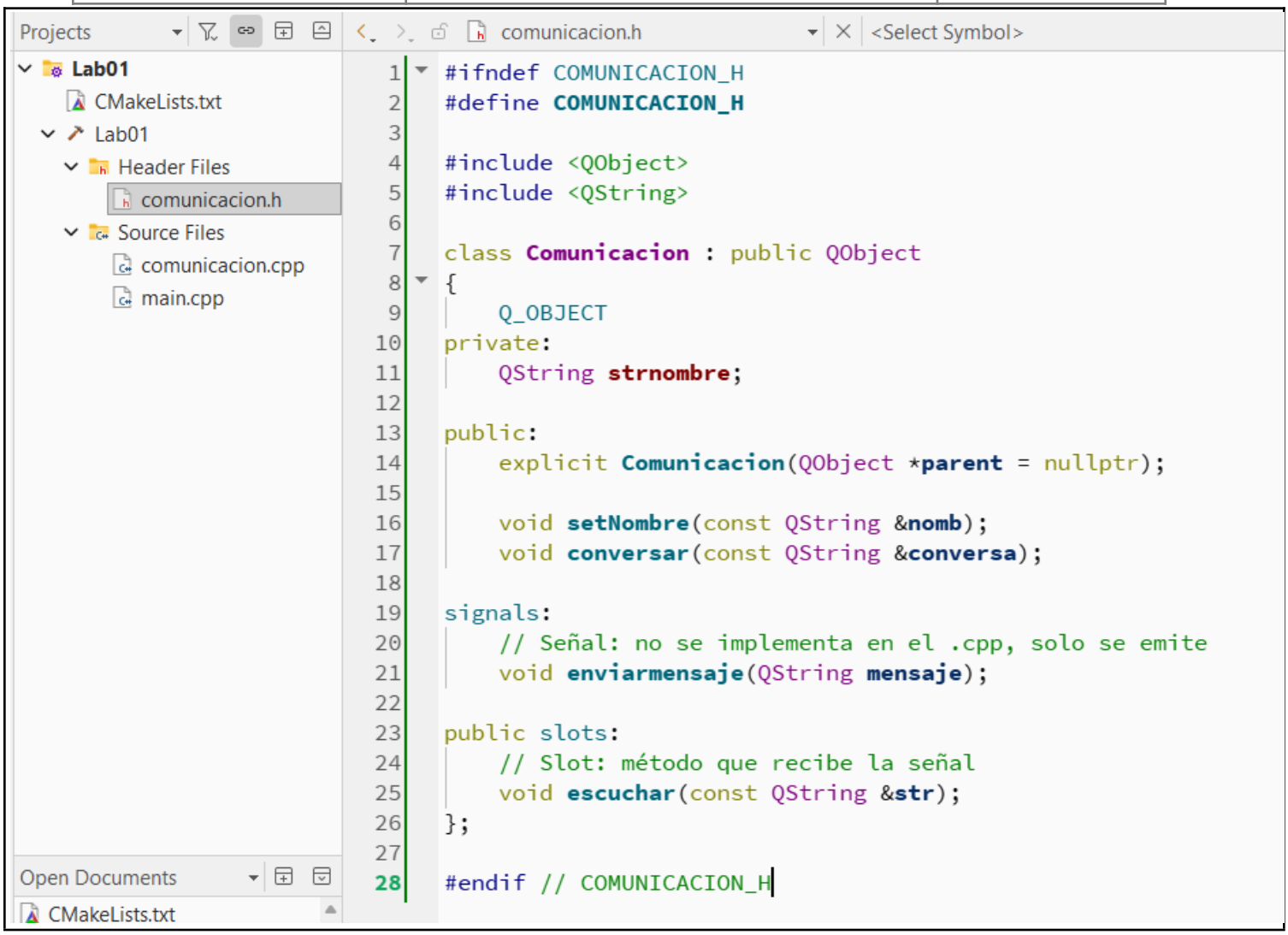
    // Creación dinámica
    Adulto *papa = new Adulto();
    papa->setNombre("Papa");
```

```
34
35      // Al pasar 'papa' en el constructor, se vuelven hijos de 'papa'
36      Adulto *hijo1 = new Adulto(papa);
37      Adulto *hijo2 = new Adulto(papa);
38      hijo1->setNombre("Manuel");
39      hijo2->setNombre("Jose");
40
41      // Se destruye ÚNICAMENTE al padre
42      delete papa;
43
44      return 0;
45  }
```

```
23:07:00: Starting C:\Users\User\OneDrive\Documentos\Lab01_Primitivas\build\Deskt
..objeto destruido... "Papa"
..objeto destruido... "Manuel"
..objeto destruido... "Jose"
23:07:00: The command "C:\Users\User\OneDrive\Documentos\Lab01_Primitivas\build\D
successfully.
```

Se demostró la gestión automática de memoria en árbol de Qt. Al invocar explícitamente delete papa, el destructor de QObject recorre su lista interna de children() y procede a eliminarlos antes de destruir al padre. Esto garantiza la prevención de fugas de memoria sin necesidad de destruir manualmente cada puntero hijo.

**Actividad C: Chat en Consola mediante Signals & Slots**


Projects
Lab01
CMakeLists.txt
Lab01
Header Files
comunicacion.h
Source Files
comunicacion.cpp
main.cpp
Open Documents
CMakeLists.txt
comunicacion.h
<Select Symbol>
#ifndef COMUNICACION_H
#define COMUNICACION_H
#include <QObject>
#include <QString>
class Comunicacion : public QObject
{
Q_OBJECT
private:
QString strnombre;
public:
explicit Comunicacion(QObject *parent = nullptr);
void setNombre(const QString &nomb);
void conversar(const QString &conversa);
signals:
// Señal: no se implementa en el .cpp, solo se emite
void enviarmensaje(QString mensaje);
public slots:
// Slot: método que recibe la señal
void escuchar(const QString &str);
};
#endif // COMUNICACION_H

Projects

- Lab01
  - CMakeLists.txt
  - Lab01
    - Header Files
      - comunicacion.h
    - Source Files
      - comunicacion.cpp
      - main.cpp

comunicacion.cpp — Comunicacion::escuchar(const QString &) -> void

```cpp
#include "Comunicacion.h"
#include <QDebug>

Comunicacion::Comunicacion(QObject *parent) : QObject(parent)
{
}

void Comunicacion::setNombre(const QString &nomb)
{
    strnombre = nomb;
}

void Comunicacion::escuchar(const QString &str)
{
    qDebug() << strnombre << "estoy escuchando...:" << str;
}

void Comunicacion::conversar(const QString &conversa)
{
    // Emite la señal que será capturada por el slot conectado
    emit enviarmensaje(conversa);
}
```

Lab01

- CMakeLists.txt
- Lab01
  - Header Files
    - comunicacion.h
  - Source Files
    - comunicacion.cpp
    - main.cpp

```cpp
#include <QCoreApplication>
#include "Comunicacion.h"

int main(int argc, char *argv[])
{
    QCoreApplication a(argc, argv);

    QObject *person = new QObject();
    Comunicacion *pepe = new Comunicacion(person);
    Comunicacion *jose = new Comunicacion(person);

    pepe->setNombre("Pepe");
    jose->setNombre("Jose");

    // Conexión sintaxis moderna (o con macros SIGNAL/SLOT clásicas del PDF)
    QObject::connect(pepe, &Comunicacion::enviarmensaje, jose, &Comunicacion::escuchar);
    QObject::connect(jose, &Comunicacion::enviarmensaje, pepe, &Comunicacion::escuchar);

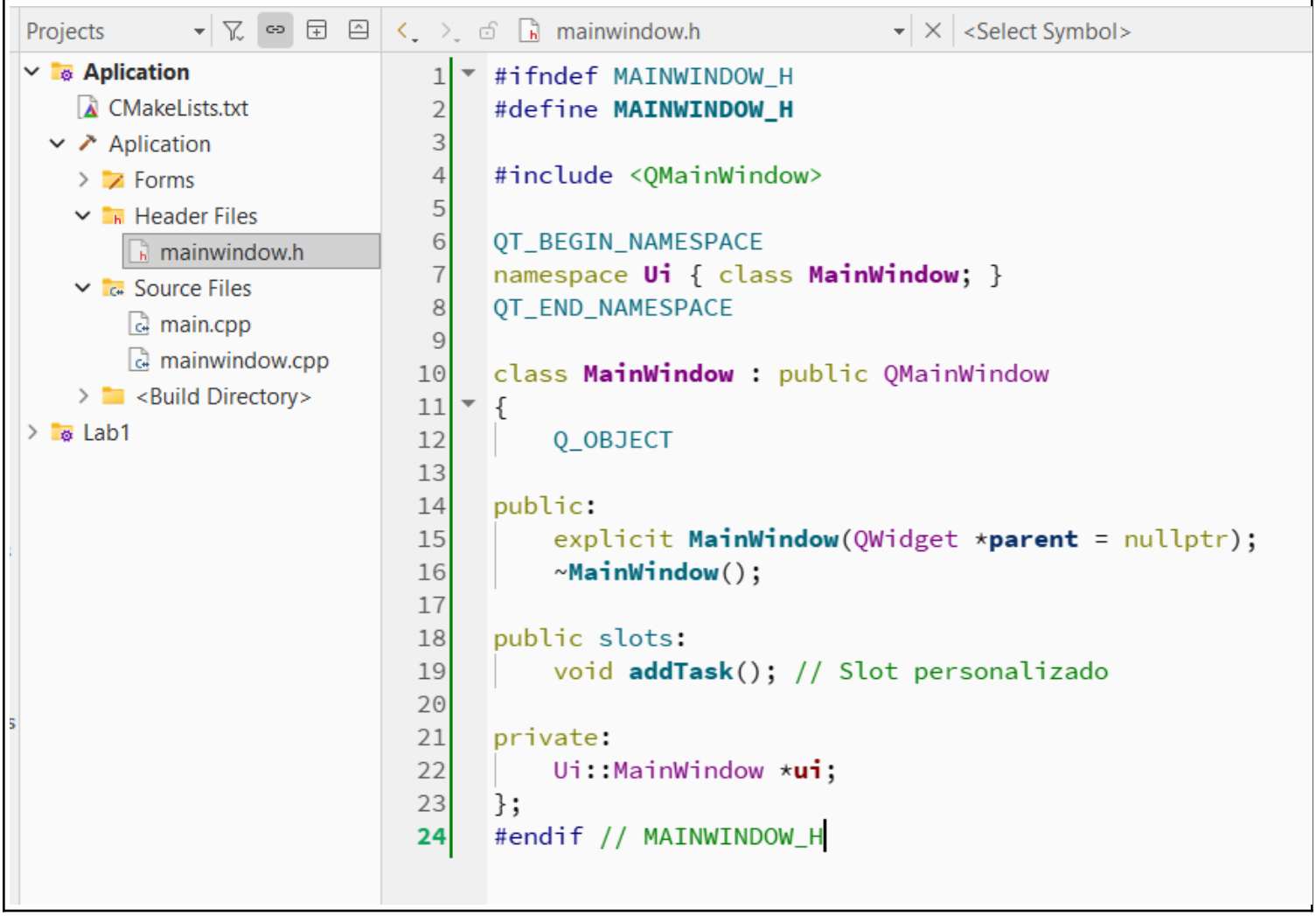
    // Prueba de emisión
    pepe->conversar("buenos dias");
    jose->conversar("como vas?");

    delete person; // Libera person, pepe y jose
    return 0;
}
```

```
23:19:13: Starting C:\Users\User\OneDrive\Documentos\L
"Jose" estoy escuchando...: "buenos dias"
"Pepe" estoy escuchando...: "como vas?"
23:19:14: The command "C:\Users\User\OneDrive\Document
```

Se estableció un mecanismo de comunicación desacoplada. Los emisores (pepe y jose) desconocen quién procesará el mensaje; únicamente ejecutan emit enviarmensaje(...). El método QObject::connect() actúa como intermediario registrando qué ranura (escuchar) debe reaccionar ante la activación del evento.

**Actividad D: Formulario GUI y Slots Personalizados**



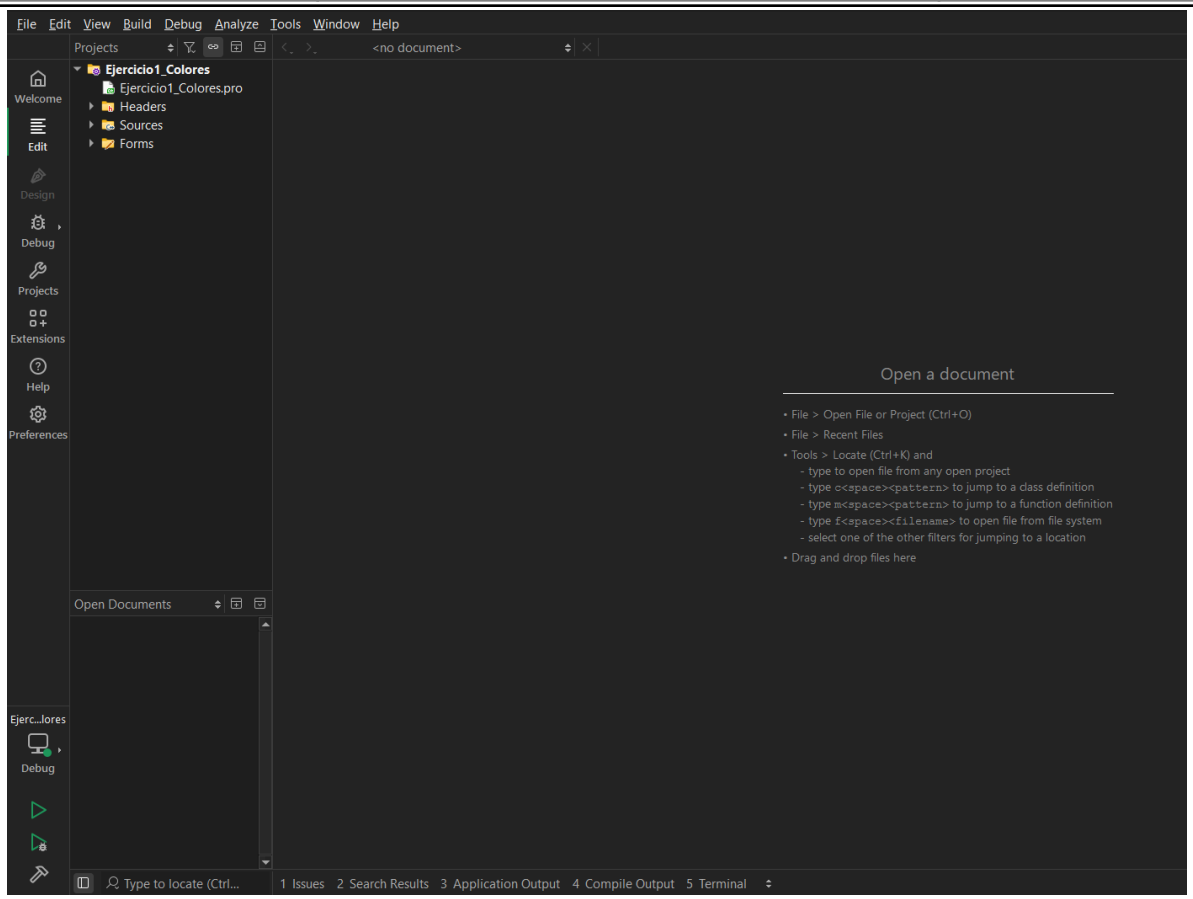



Se integró la interfaz gráfica creada con Qt Designer con el código fuente en C++. Al vincular la señal clicked del botón addTaskButton con el slot propio addTask(), logramos ejecutar una acción definida en la clase receptora cada vez que el usuario presiona el botón en la GUI.

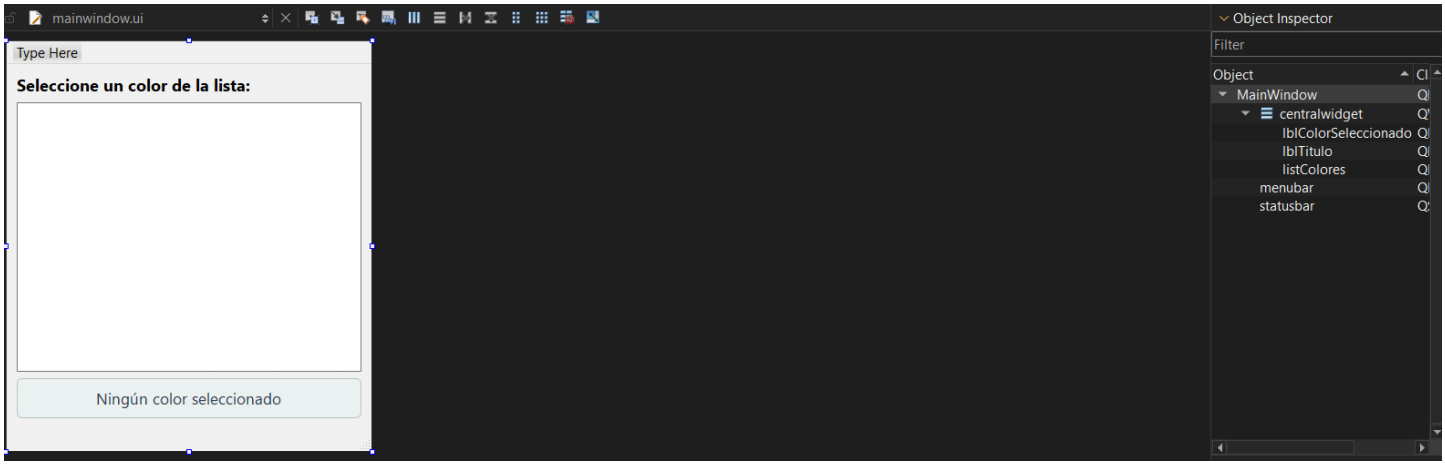
## Problema Propuesto:

### Ejercicio 1: Comunicación entre Task.ui y MainWindow mediante Signals & Slots

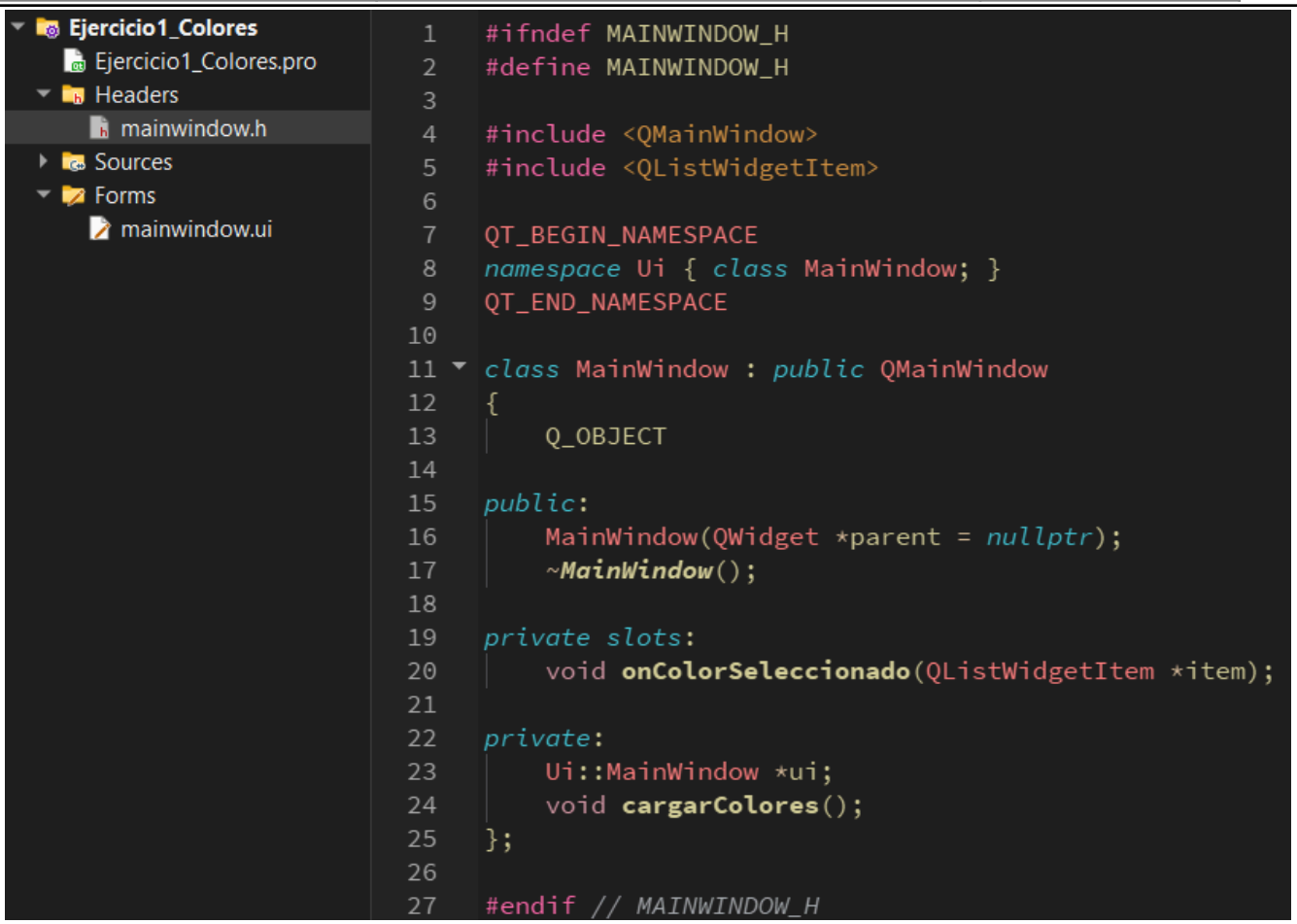
**Enunciado:** Crear una interface gráfica (que implemente señales y slots) que muestre una lista de nombres de colores, al dar clic sobre alguno que se muestre un Label o un Text con el nombre del color.



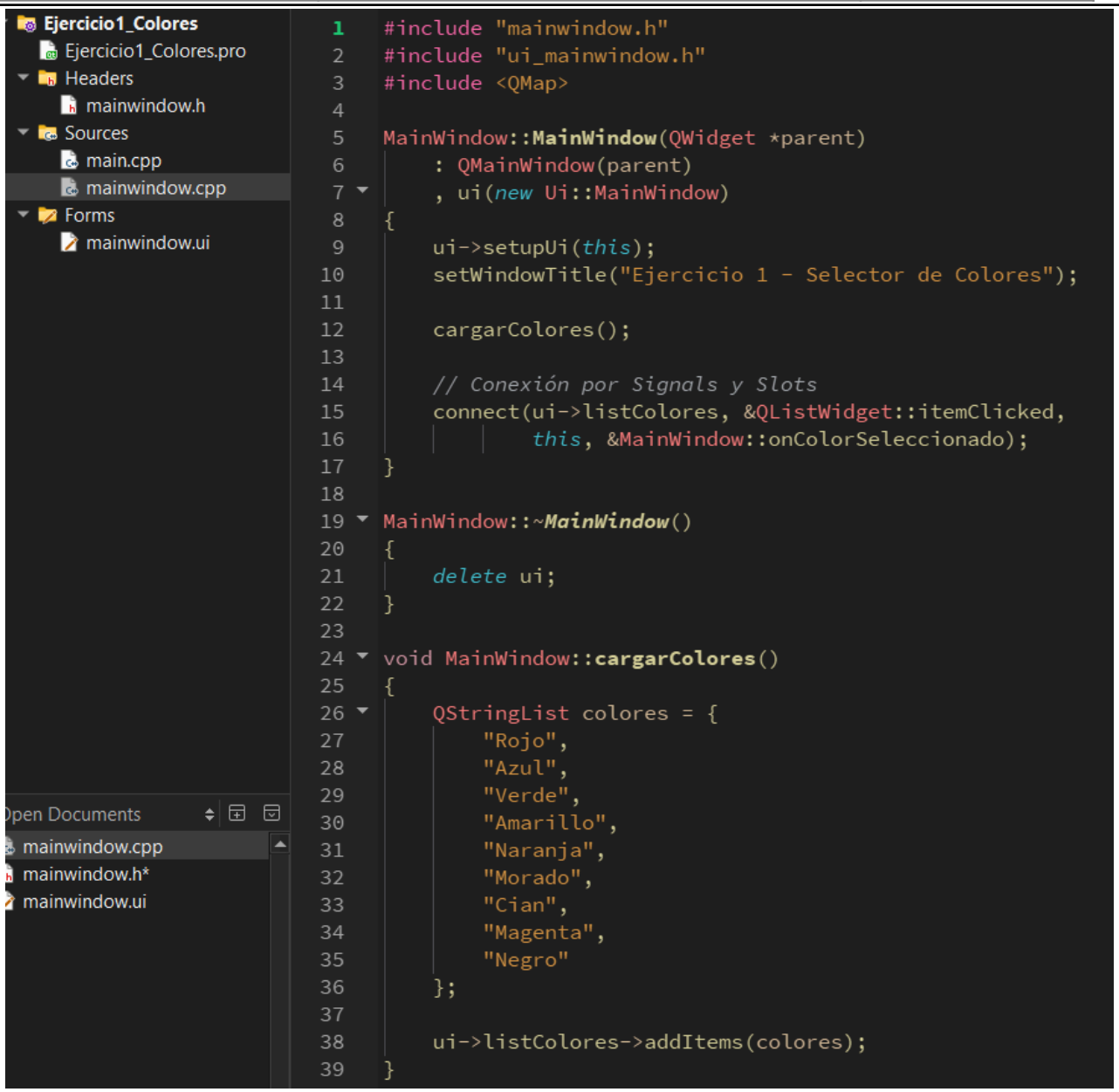

Se configuró el proyecto en Qt Creator seleccionando el kit Desktop Qt 6.8.3 MinGW 64-bit bajo qmake, inicializando la estructura base con los archivos de cabecera, código fuente y formulario de interfaz.



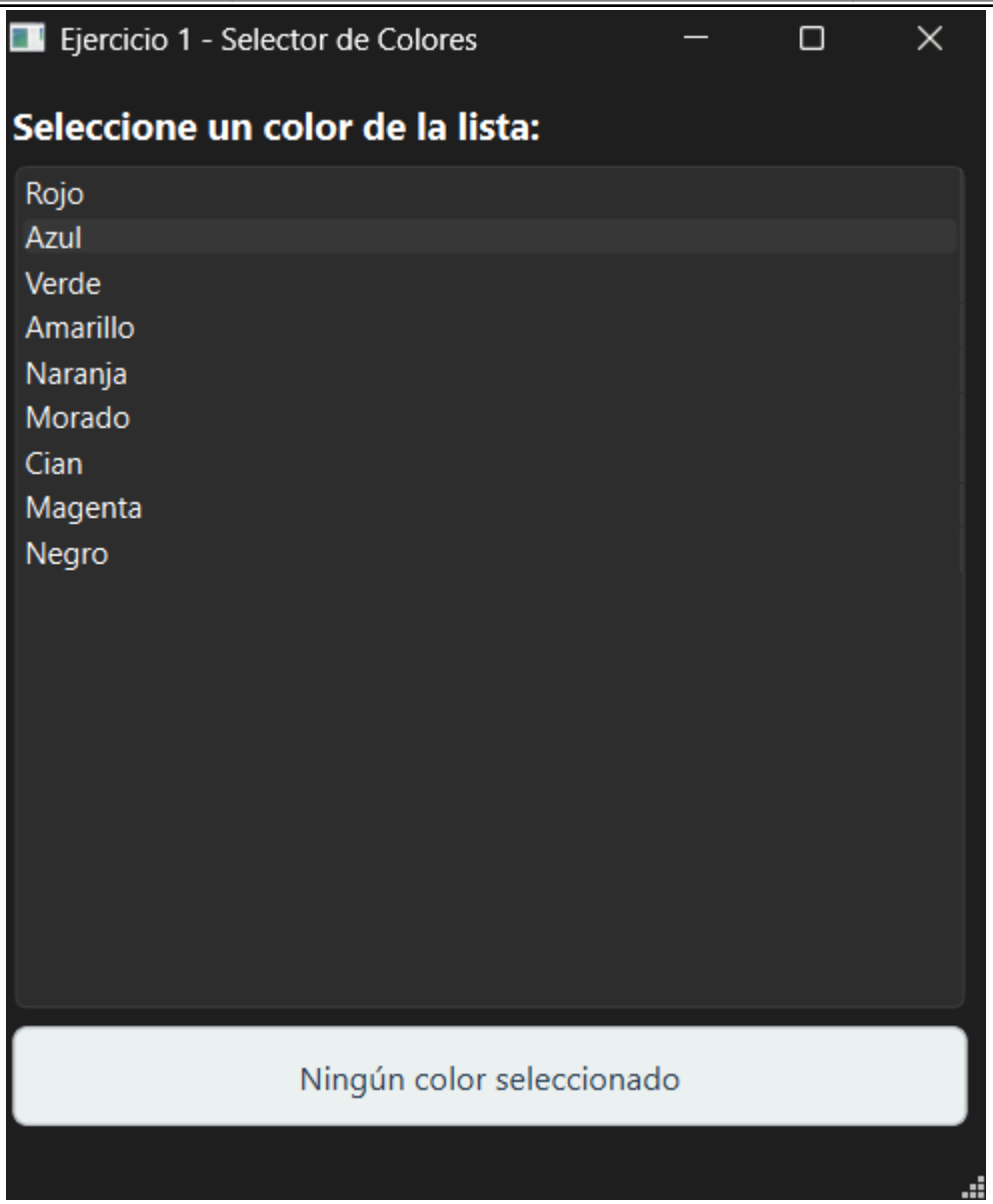

Se diseñó la interfaz gráfica incorporando un control QListWidget denominado listColores y una etiqueta QLabel lblColorSeleccionado, organizados mediante un layout vertical para asegurar una distribución ordenada y adaptable.

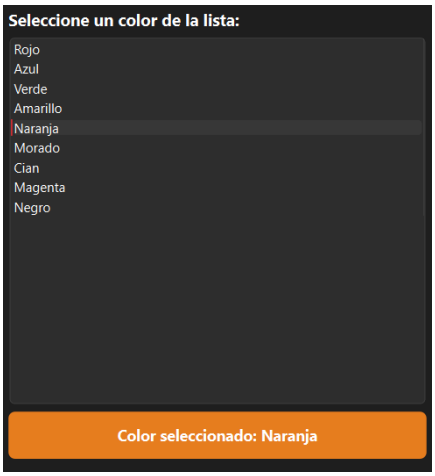


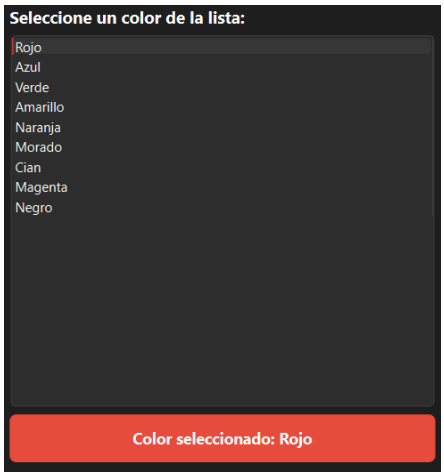

En la cabecera mainwindow.h se declaró el slot privado onColorSeleccionado, el cual recibe la referencia al elemento seleccionado en la lista para procesar el evento de interacción.

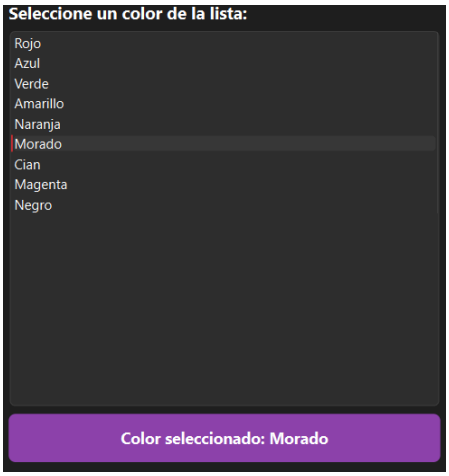



En mainwindow.cpp se enlazó la señal itemClicked del componente con el slot correspondiente mediante la función connect(). Asimismo, se implementó la lógica que actualiza el texto y la apariencia del QLabel en tiempo de ejecución.

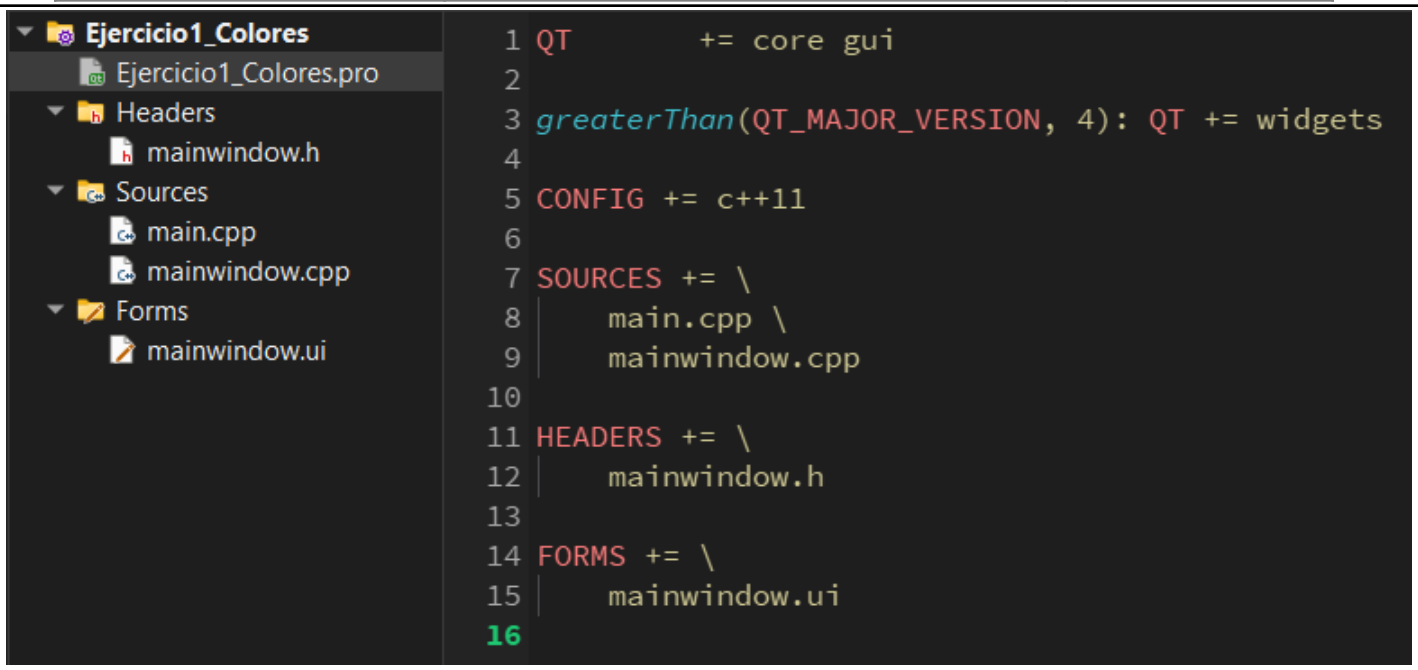


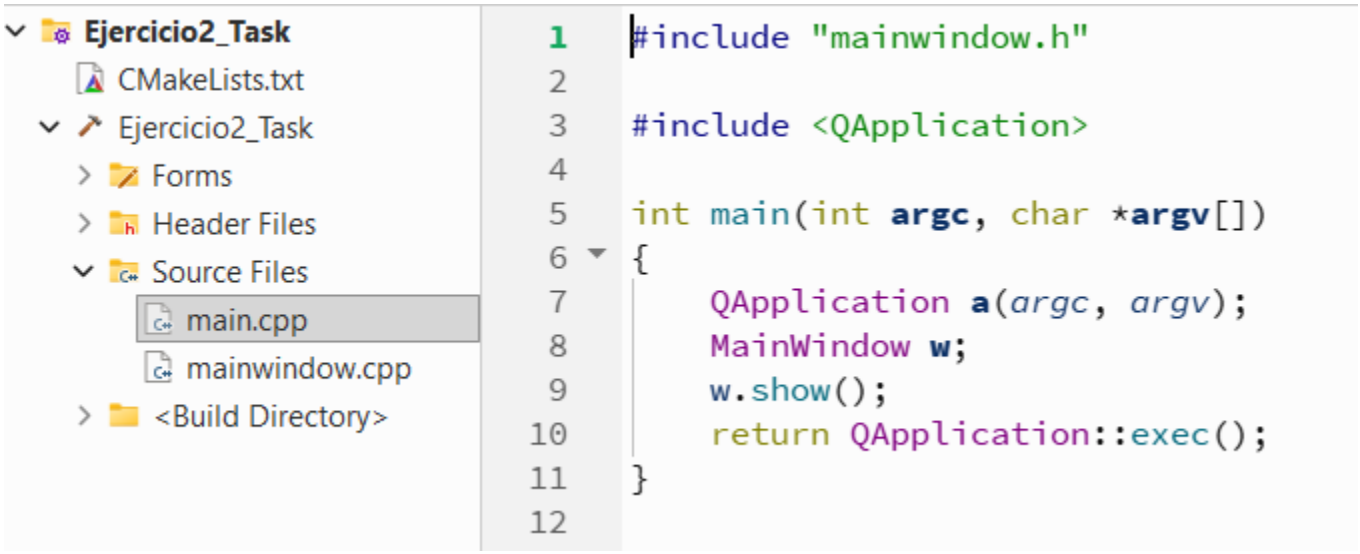

Ejecución del programa. Se constató la correcta emisión de la señal ante el clic del usuario y la respuesta del slot, reflejando de forma inmediata el color seleccionado en la interfaz.



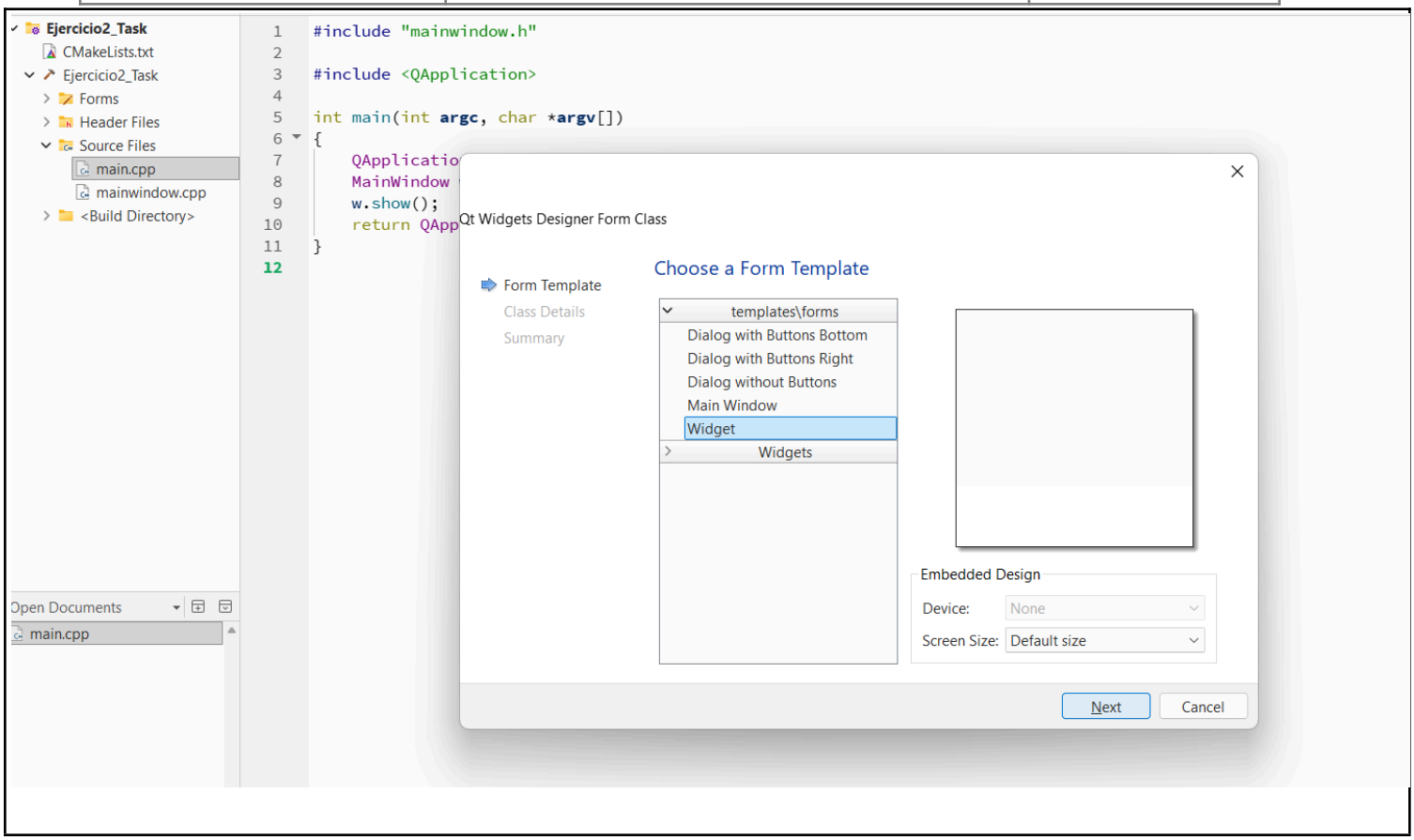

Configuración del archivo del proyecto (.pro), donde se especifican los módulos de Qt requeridos (core, gui y widgets) y se referencian los componentes del código fuente para el proceso de compilación con qmake.

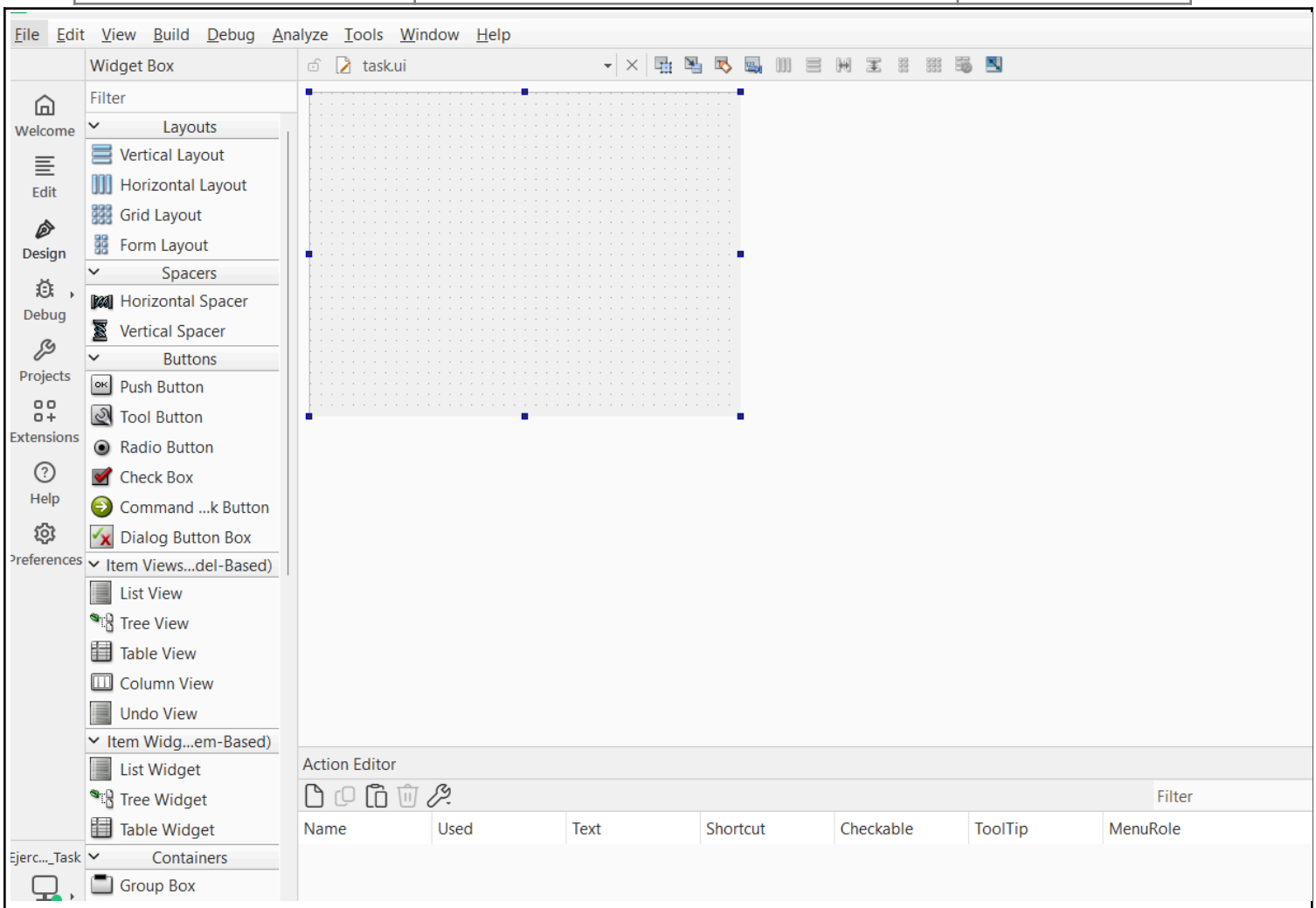
**Ejercicio 2: Comunicación entre Task.ui y MainWindow mediante Signals & Slots**

**Enunciado:** Averiguar la implementación por signals y slots para comunicar los objetos del formulario de Task.ui (por ejemplo una caja de texto o un checkbox/label) desde MainWindow.h y CPP.

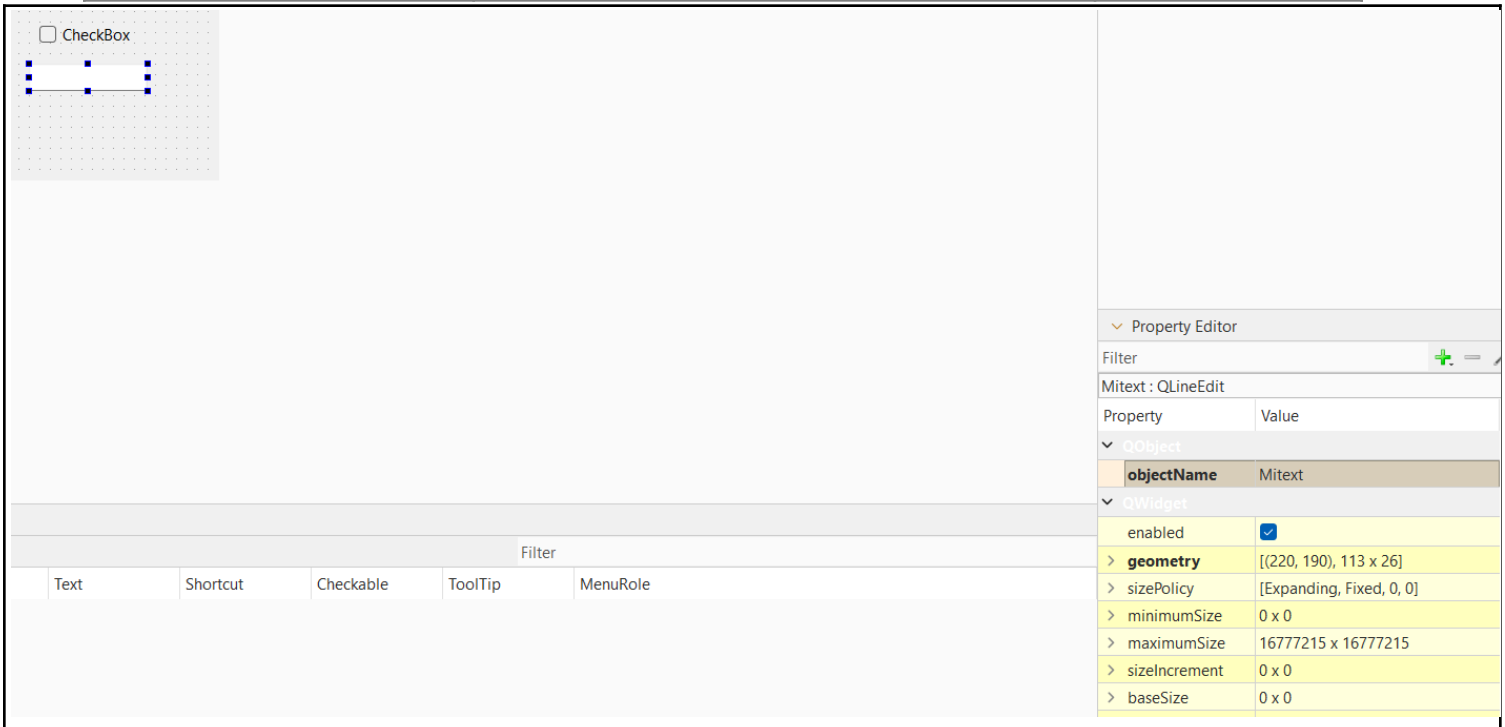


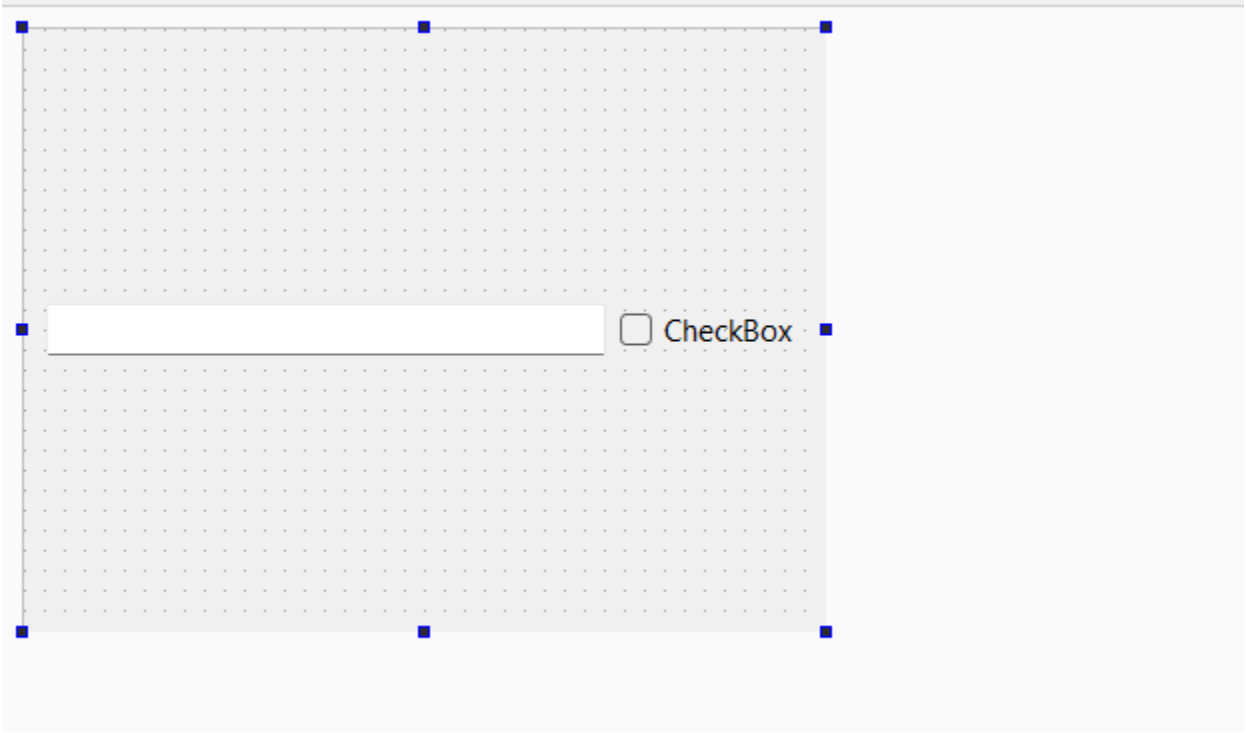

Se creó un proyecto de tipo **Qt Widgets Application** empleando el sistema de compilación **qmake**. La clase base generada por defecto es MainWindow heredando de QMainWindow, la cual servirá como ventana contenedora principal de la aplicación y gestionará el ciclo de vida de los widgets secundarios.


Ejercicio2_Task
CMakeLists.txt
Ejercicio2_Task
Forms
Header Files
Source Files
main.cpp
mainwindow.cpp
<Build Directory>
Open Documents
main.cpp
#include "mainwindow.h"
#include <QApplication>
int main(int argc, char *argv[])
{
QApplicatio
MainWindow
w.show();
return QApp
}
Qt Widgets Designer Form Class
Choose a Form Template
Form Template
Class Details
Summary
templates\forms
Dialog with Buttons Bottom
Dialog with Buttons Right
Dialog without Buttons
Main Window
Widget
Widgets
Embedded Design
Device:
None
Screen Size:
Default size
Next
Cancel




Para cumplir con el diseño modular, se agregó una nueva clase con formulario visual denominada Task, basada en la plantilla QWidget. Esta clase encapsulará el diseño visual y el comportamiento de una tarea individual de forma independiente a la ventana principal.

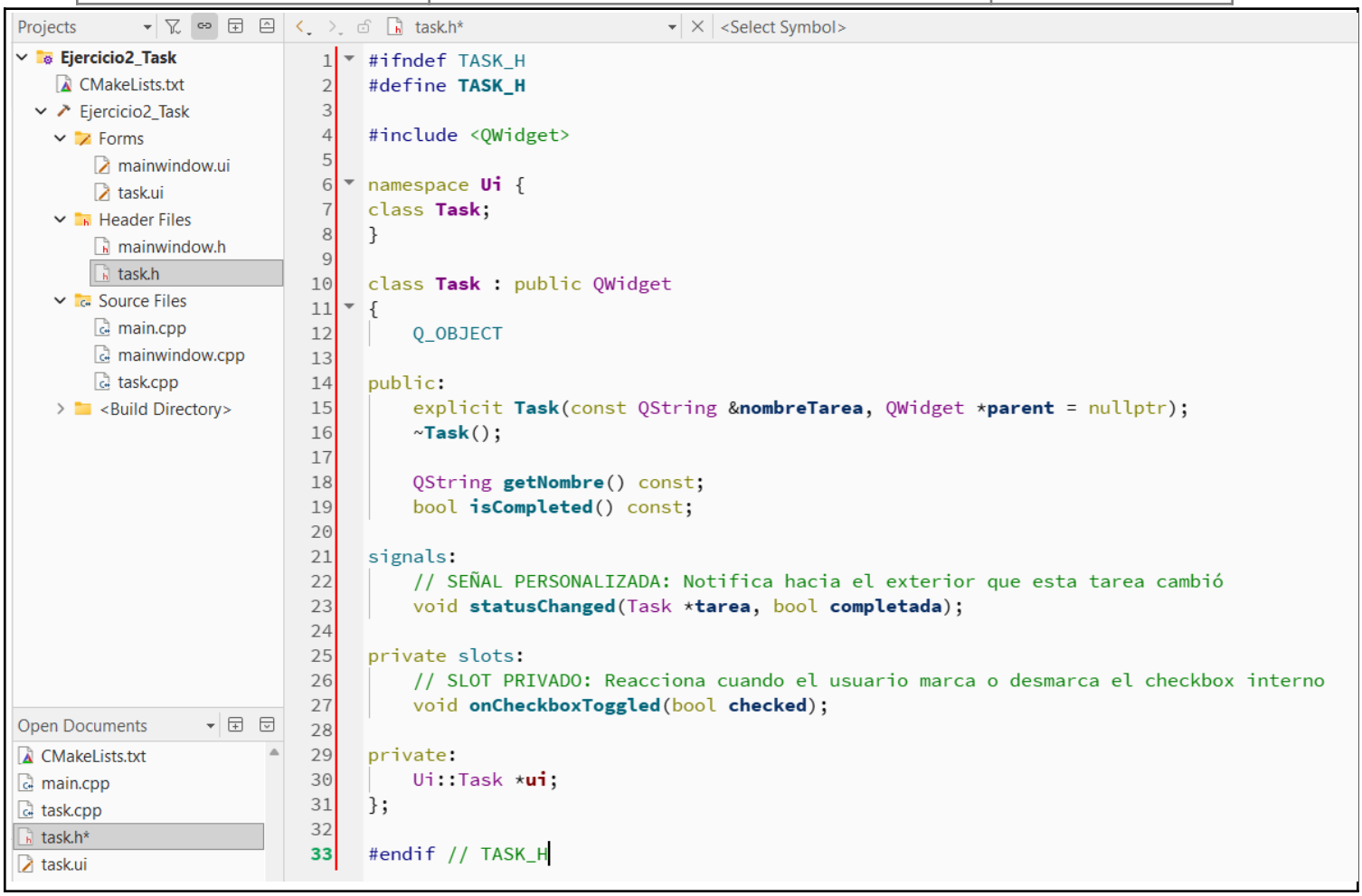


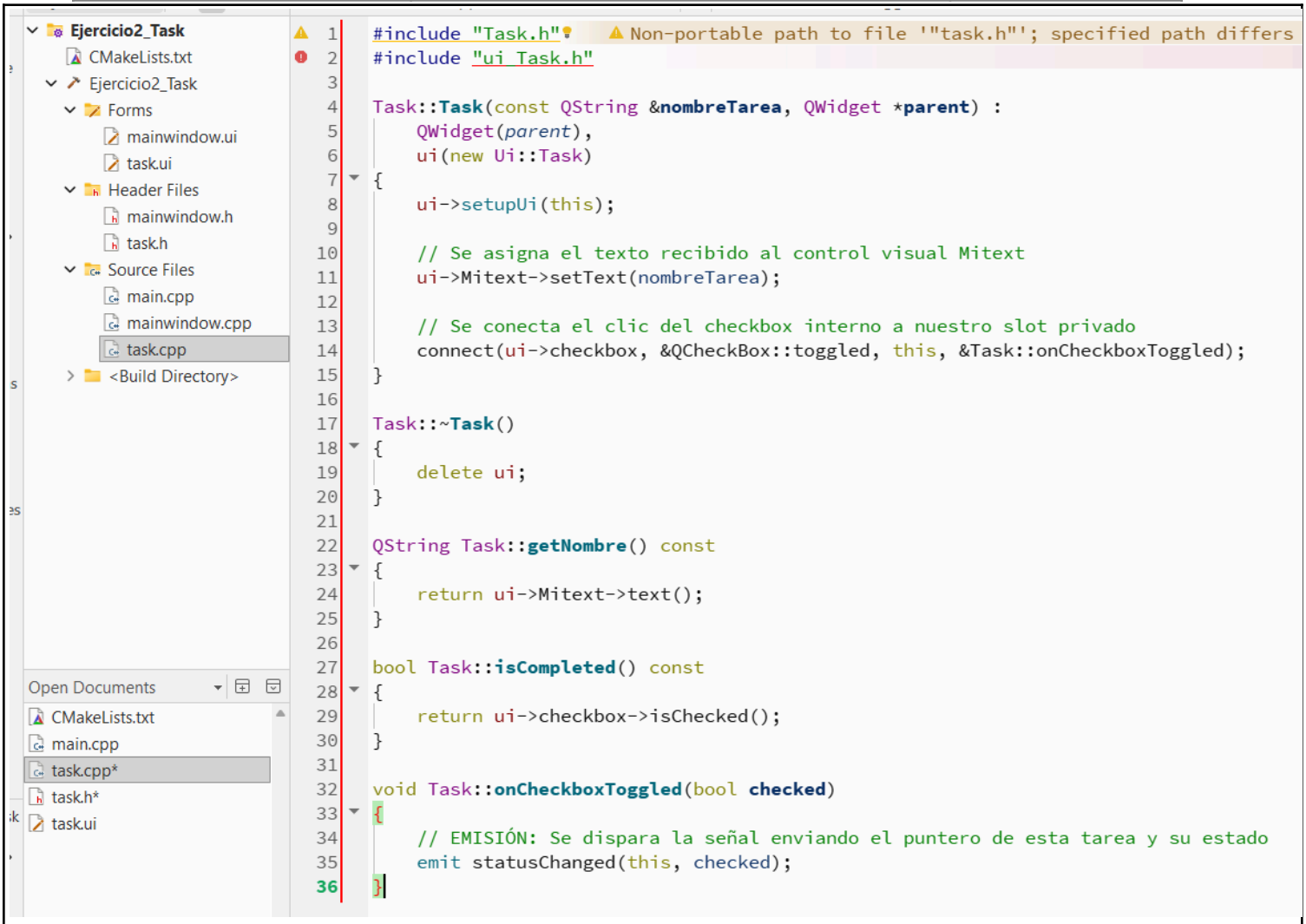




En el diseñador visual de Qt Designer se integraron dos controles fundamentales dentro de un contenedor horizontal (QHBoxLayout):

- QCheckBox (con objectName = checkbox): Control que representa el estado de finalización de la tarea.
- QLineEdit (con objectName = Mitext): Campo que contendrá la descripción textual de la tarea.

Este diseño permite instanciar de forma compacta múltiples filas de tareas dentro de la aplicación.

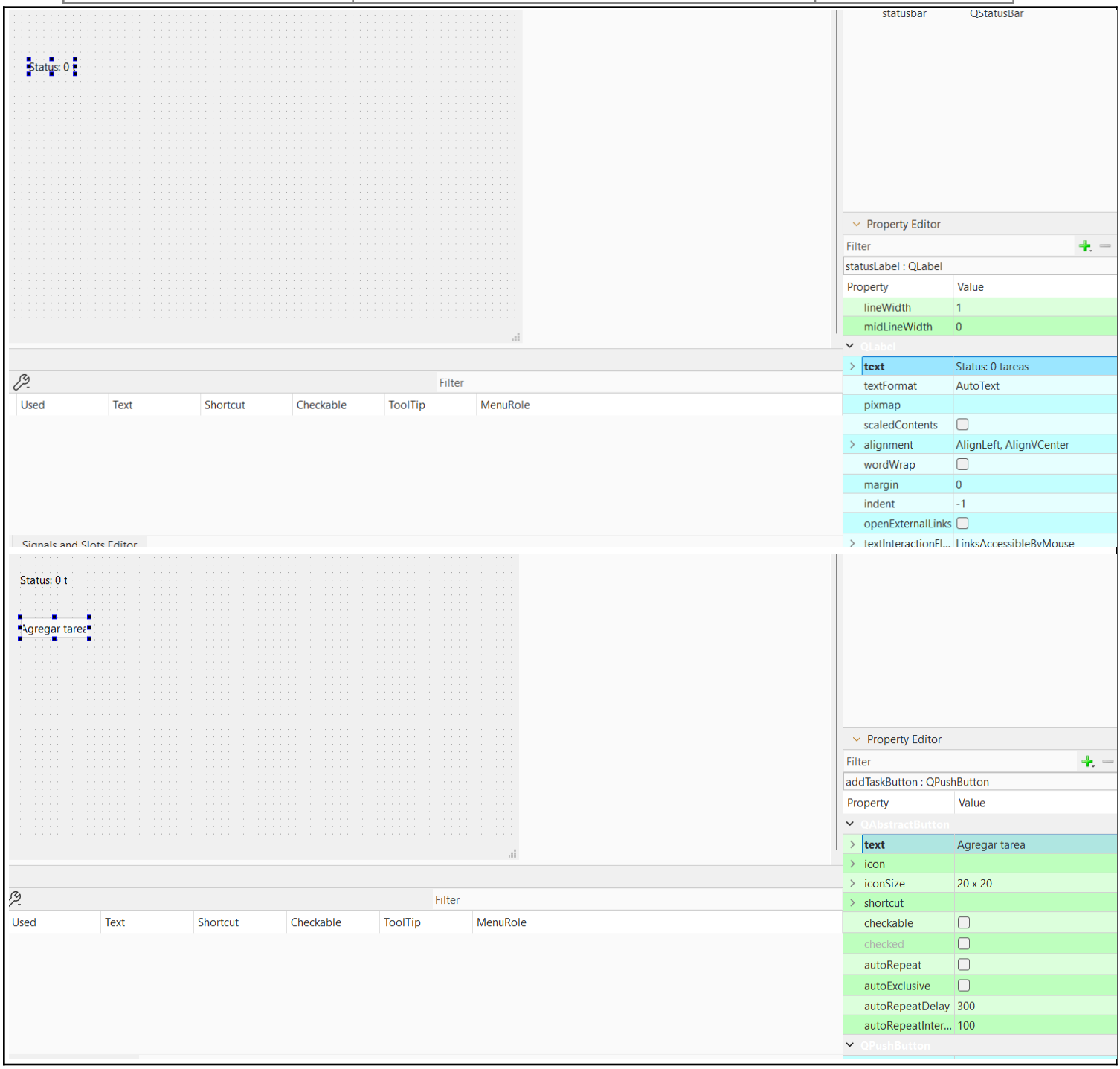

Projects
Ejercicio2_Task
CMakeLists.txt
Ejercicio2_Task
Forms
mainwindow.ui
task.ui
Header Files
mainwindow.h
task.h
Source Files
main.cpp
mainwindow.cpp
task.cpp
<Build Directory>
Open Documents
CMakeLists.txt
main.cpp
task.cpp
task.h*
task.ui
task.h*
<Select Symbol>
#ifndef TASK_H
#define TASK_H
#include <QWidget>
namespace Ui {
class Task;
}
class Task : public QWidget
{
Q_OBJECT
public:
explicit Task(const QString &nombreTarea, QWidget *parent = nullptr);
~Task();
QString getNombre() const;
bool isCompleted() const;
signals:
// SEÑAL PERSONALIZADA: Notifica hacia el exterior que esta tarea cambió
void statusChanged(Task *tarea, bool completada);
private slots:
// SLOT PRIVADO: Reacciona cuando el usuario marca o desmarca el checkbox interno
void onCheckboxToggled(bool checked);
private:
Ui::Task *ui;
};
#endif // TASK_H

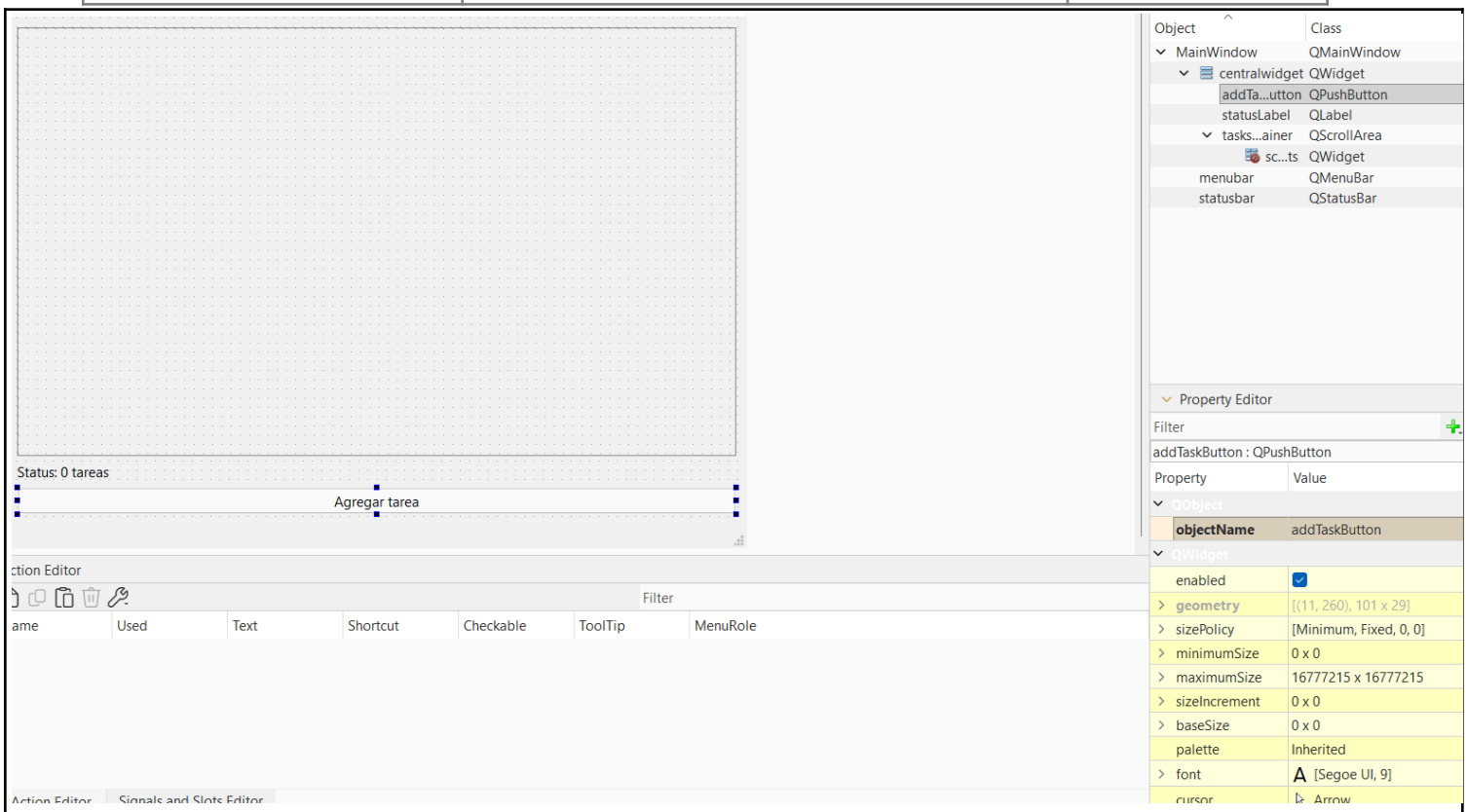



Para comunicar los eventos de Task.ui sin violar el encapsulamiento de datos, se declaró en Task.h la señal propia:

```
C/C++
signals:
    void statusChanged(Task *tarea, bool completada);
```

Dentro del constructor de Task.cpp, se vinculó la señal nativa &QCheckBox::toggled con la función miembro onCheckboxToggled(). Al interactuar con la casilla, este método ejecuta la macro emit statusChanged(this, checked). De este modo, la tarea actúa como un emisor autónomo que notifica su estado al entorno sin requerir conocer qué clase recibirá la notificación.

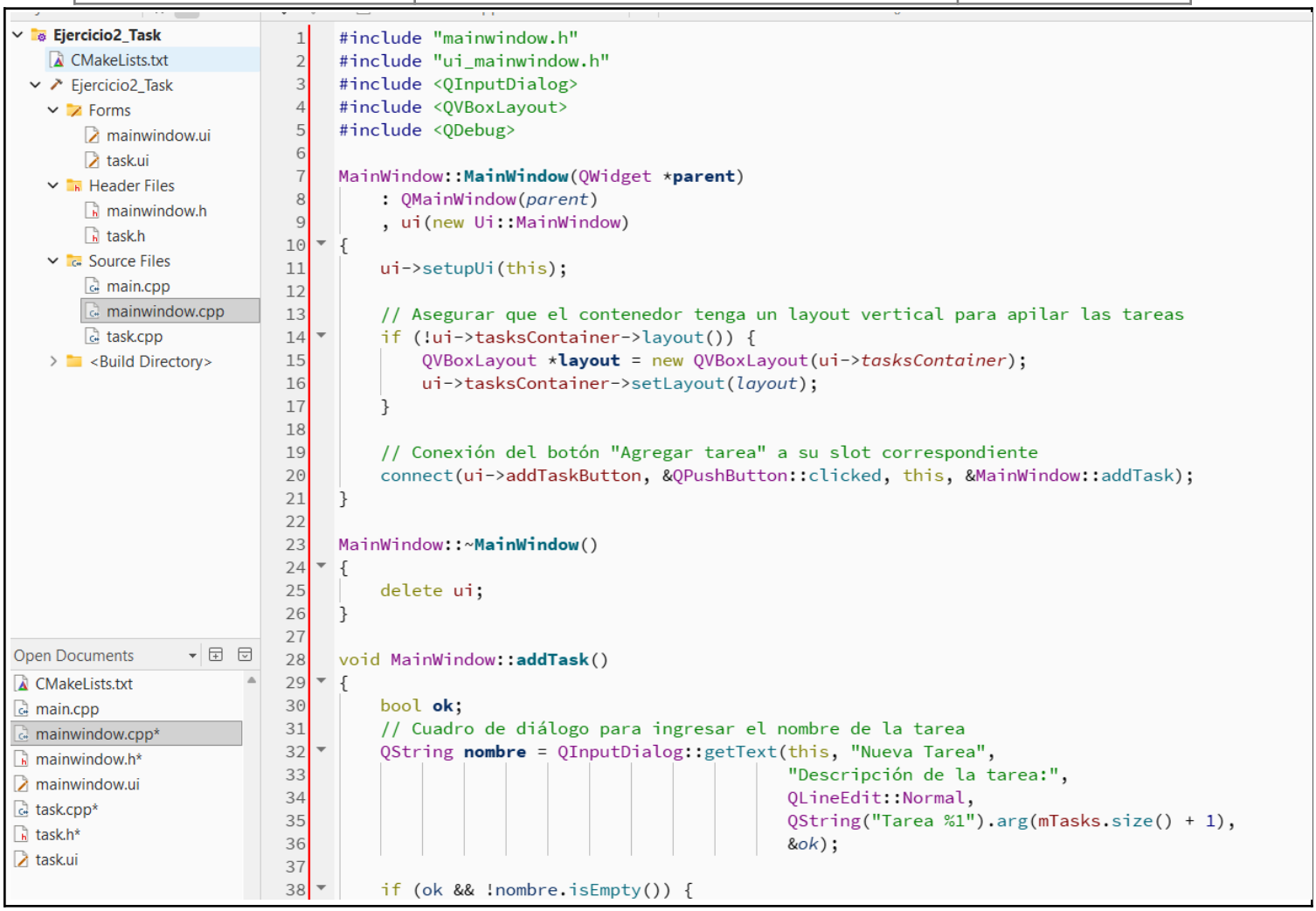

Status: 0
statusbar
QStatusBar
Property Editor
Filter
statusLabel : QLabel
Property
Value
lineWidth
1
midLineWidth
0
QLabel
text
Status: 0 tareas
textFormat
AutoText
pixmap
scaledContents
alignment
AlignLeft, AlignVCenter
wordWrap
margin
0
indent
-1
openExternalLinks
Used
Text
Shortcut
Checkable
ToolTip
MenuRole
Status: 0 t
Agregar tarea
addTaskButton : QPushButton
QAbstractButton
Agregar tarea
icon
iconSize
20 x 20
shortcut
checkable
checked
autoRepeat
autoExclusive
autoRepeatDelay
300
autoRepeatInter...
100
QPushButton

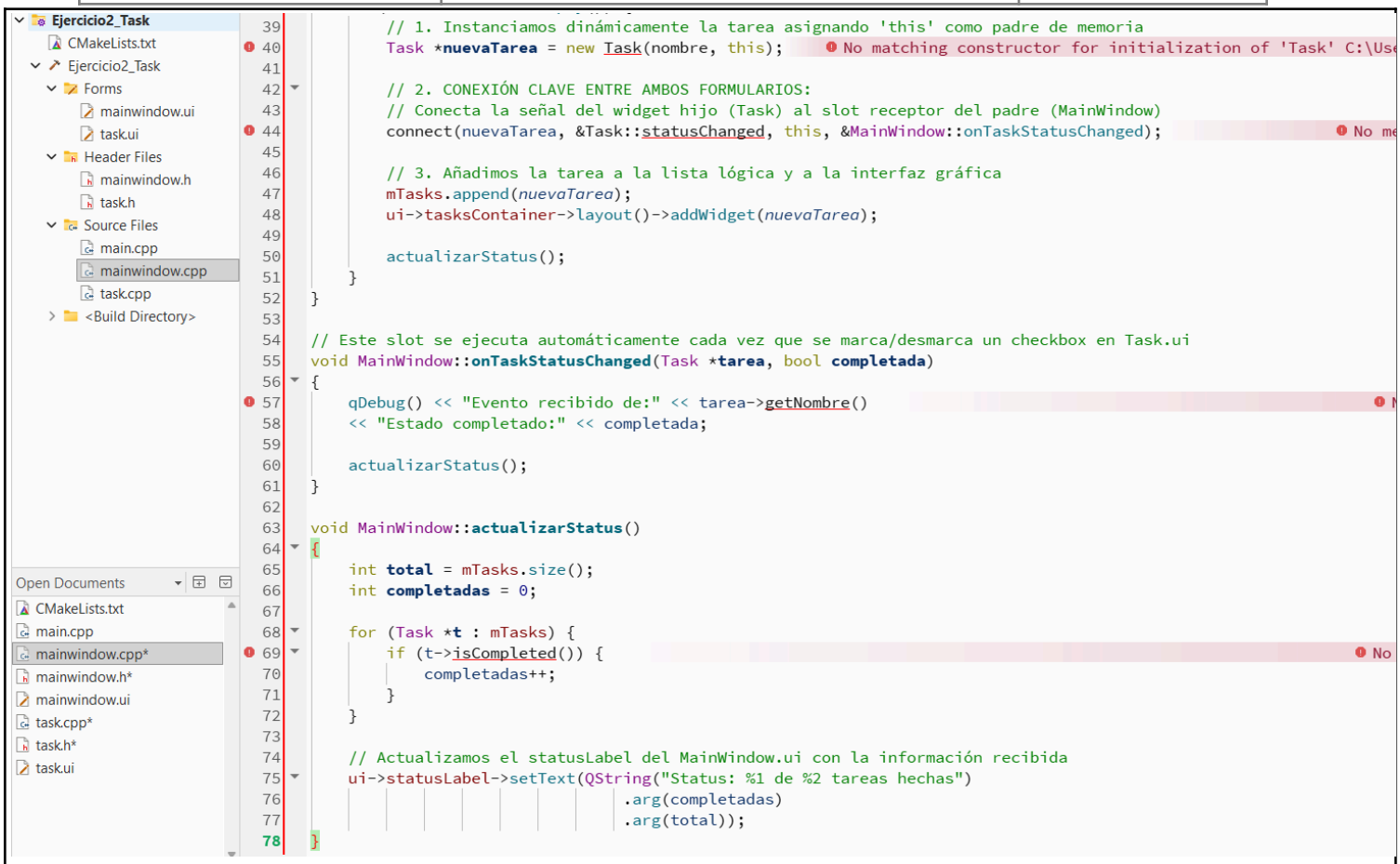



Siguiendo las pautas del laboratorio (Págs. 7 y 9), se agregaron los componentes de control en la ventana principal:

- addTaskButton (QPushButton): Disparador para crear nuevas instancias de tareas.
- statusLabel (QLabel): Etiqueta informativa que reflejará en tiempo real el conteo de tareas completadas.
- tasksContainer: Área con disposición vertical destinada a albergar dinámicamente los widgets de tipo Task.


Ejercicio2_Task
CMakeLists.txt
Ejercicio2_Task
Forms
mainwindow.ui
task.ui
Header Files
mainwindow.h
task.h
Source Files
main.cpp
mainwindow.cpp
task.cpp
<Build Directory>
Open Documents
CMakeLists.txt
main.cpp
mainwindow.h*
#ifndef MAINWINDOW_H
#define MAINWINDOW_H
#include <QMainWindow>
#include <QVector>
#include "Task.h" // Inclusión indispensable para reconocer la clase Task
Non-portable
QT_BEGIN_NAMESPACE
namespace Ui { class MainWindow; }
QT_END_NAMESPACE
class MainWindow : public QMainWindow
{
Q_OBJECT
public:
explicit MainWindow(QWidget *parent = nullptr);
~MainWindow();
public slots:
void addTask(); // Slot para instanciar una nueva tarea
void onTaskStatusChanged(Task *tarea, bool completada); // Slot receptor de Task.ui
private:
Ui::MainWindow *ui;
QVector<Task*> mTasks; // Vector que almacena las tareas creadas
void actualizarStatus(); // Función que actualiza el texto del statusLabel
};
#endif // MAINWINDOW_H


Ejercicio2_Task
CMakeLists.txt
Ejercicio2_Task
Forms
mainwindow.ui
task.ui
Header Files
mainwindow.h
task.h
Source Files
main.cpp
mainwindow.cpp
task.cpp
<Build Directory>
Open Documents
CMakeLists.txt
main.cpp
mainwindow.cpp*
mainwindow.h*
mainwindow.ui
task.cpp*
task.h*
task.ui
#include "mainwindow.h"
#include "ui_mainwindow.h"
#include <QInputDialog>
#include <QVBoxLayout>
#include <QDebug>
MainWindow::MainWindow(QWidget *parent)
: QMainWindow(parent)
, ui(new Ui::MainWindow)
{
ui->setupUi(this);
// Asegurar que el contenedor tenga un layout vertical para apilar las tareas
if (!ui->tasksContainer->layout()) {
QVBoxLayout *layout = new QVBoxLayout(ui->tasksContainer);
ui->tasksContainer->setLayout(layout);
}
// Conexión del botón "Agregar tarea" a su slot correspondiente
connect(ui->addTaskButton, &QPushButton::clicked, this, &MainWindow::addTask);
}
MainWindow::~MainWindow()
{
delete ui;
}
void MainWindow::addTask()
{
bool ok;
// Cuadro de diálogo para ingresar el nombre de la tarea
QString nombre = QInputDialog::getText(this, "Nueva Tarea",
"Descripción de la tarea:",
QLineEdit::Normal,
QString("Tarea %1").arg(mTasks.size() + 1),
&ok);
if (ok && !nombre.isEmpty()) {




En mainwindow.cpp se implementó la conexión entre el widget secundario y la ventana principal. Dentro del método addTask(), en el instante en que se crea una nueva instancia de Task, se establece el enlace mediante:

```
C/C++
connect(nuevaTarea, &Task::statusChanged, this, &MainWindow::onTaskStatusChanged);
```

Cuando cualquier tarea emite su señal statusChanged, MainWindow ejecuta inmediatamente su slot onTaskStatusChanged(), recalculando el total de ítems completados y actualizando la propiedad text de su propio statusLabel. Asimismo, al pasar this al constructor de Task, se asegura que la memoria de las tareas creadas esté subordinada al ciclo de vida de MainWindow.




Al ejecutar el programa, se realizaron pruebas de interacción dinámica:

1. Se añadieron tareas mediante el diálogo de entrada, verificando que los widgets de Task.ui se incrustan correctamente en el layout de MainWindow.
2. Al activar la casilla de verificación (QCheckBox) de una tarea, se comprobó la emisión instantánea de la señal hacia el slot de la ventana principal, reflejando el cambio numérico en el statusLabel sin congelamientos de la interfaz ni errores de segmentación.
3. En la consola de depuración se verificó la trazabilidad de los datos mediante los registros de qDebug().

## III. CONCLUSIONES Y RETROALIMENTACIÓN GENERAL

### Conclusiones del Laboratorio

- **Gestión de Memoria Efectiva:** El sistema de objetos padre-hijo de QObject optimiza la administración de memoria en C++, mitigando fugas (*memory leaks*) mediante destrucciones automáticas recursivas.
- **Bajo Acoplamiento con Signals & Slots:** El paradigma de comunicación entre objetos mediante señales y ranuras evita dependencias directas entre componentes, facilitando que emisores y receptores operen de manera modular y escalable.
- **Productividad en GUI:** La combinación de Qt Designer junto con clases de formulario (.ui, .h, .cpp) separa de forma clara la presentación visual de la lógica de negocio, reduciendo tiempos de desarrollo y mantenimiento de interfaces complejas.

### Retroalimentación General

- **Observación sobre macros clásicas:** La guía utiliza la sintaxis clásica connect(..., SIGNAL(...), ..., SLOT(...)). Es recomendable usar la sintaxis moderna basada en punteros a funciones miembro (connect(sender, &Sender::signal, receiver, &Receiver::slot)), ya que permite la verificación de tipos en tiempo de compilación y previene fallos silenciosos por errores ortográficos.
- **Manejo de colecciones visuales:** Para el diseño de listas complejas que involucran widgets personalizados (como Task), se sugiere hacer uso de layouts verticales dinámicos (QVBoxLayout) dentro de un QScrollArea para contener las instancias generadas.

## IV. REFERENCIAS

- https://www.qt.io/
- **QT creator manual**